

Universidad de la República

Facultad de Ingeniería



Maestría en Ciencia de Datos y Aprendizaje Automático

Teoría y Algoritmia de Optimización 2024

Unrolled optimization

Ana Cortazzo
Pablo Schiavone

1 de octubre de 2024

Contenido

1. Introducción **2**

2. Marco Teórico **2**
2.1. Redes neuronales 2
2.2. *Sparse coding* o codificación esparsa 4
2.3. Métodos iterativos de optimización 7
2.3.1. Alternating Direction Method of Multipliers (ADMM) 7
2.3.2. Iterative Shrinkage-Thresholding Algorithm 9
2.3.3. Fast Iterative Shrinkage-Thresholding Algorithm 10
2.4. *Algorithm Unrolling* 11
2.4.1. Aplicaciones 12

3. Breve revisión del Estado del arte **14**

4. Aplicación **15**

5. Conclusiones **16**

Referencias **16**

1. Introducción

En los últimos años, el campo del aprendizaje automático ha experimentado un avance significativo gracias a la incorporación de técnicas de optimización en redes neuronales profundas. Estas técnicas han permitido desarrollar modelos más eficientes y robustos para diversas aplicaciones, como la clasificación de imágenes, el procesamiento de señales, y el diagnóstico de fallos. Dentro de este contexto, la *codificación esparsa* ha emergido como una herramienta clave para aprender representaciones esparsas de los datos. Técnicas más recientes como el *unrolling de algoritmos* han proporcionado interpretabilidad y mejoras en la velocidad de convergencia de las redes neuronales, y han permitido abordar problemas de alta dimensionalidad de manera efectiva.

Este trabajo se estructura en dos etapas. La primera etapa, presentada en este documento, tiene un enfoque teórico. En ella, se realiza una revisión teórica de los principales métodos de optimización iterativa, como el *Alternating Direction Method of Multipliers* (ADMM), el *Iterative Shrinkage-Thresholding Algorithm* (ISTA) y su versión acelerada, el *Fast ISTA* (ver Sección 2.3). A su vez, se introducen los fundamentos de la *codificación esparsa* (Sección 2.2) y de las redes neuronales (Sección 2.1). Finalmente, se aborda el concepto de *Algorithm Unrolling* (Sección 2.4), un método que consiste en desenrollar algoritmos iterativos dentro de la estructura de una red neuronal, permitiendo aprender parámetros de manera supervisada y obtener modelos más eficientes e interpretables. En la Sección 3, se exploran aplicaciones recientes de estas técnicas en el campo de la visión por computadora y el diagnóstico de fallos mecánicos, entre otros.

La segunda etapa del trabajo consistirá en una implementación práctica de estas técnicas, enfocada específicamente en el desenrollado de algoritmos. Se espera desarrollar un modelo basado en Python que aplique el *unrolling* a un problema específico de optimización, utilizando las bases teóricas desarrolladas en este informe. De esta manera, se busca no solo entender los fundamentos de las técnicas discutidas, sino también demostrar su aplicabilidad y eficiencia en un entorno real.

2. Marco Teórico

2.1. Redes neuronales

Las *redes neuronales artificiales* (ANNs por sus siglas en inglés*Artificial Neural Networks*) son un conjunto de algoritmos de aprendizaje automático, cuyo objetivo es reconocer patrones complejos en los datos. El estudio de las redes neuronales ha sido parcialmente inspirado por la observación de que los sistemas de aprendizaje biológicos están formados por redes complejas de neuronas interconectadas. De manera análoga, las ANNs están compuestas por neuronas artificiales, que son funciones matemáticas conectadas a través de capas, donde cada unidad recibe entradas con valores reales y produce una salida real, que a su vez puede servir como entrada para otras unidades [Mitchell, 1997].

Sea x un vector de entrada:

$$x \in \mathbb{R}^n = a^{(1)} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

Para una red de una sola neurona, la función objetivo (o hipótesis) será:

$$h_\theta(x) = a^{(2)} \in \mathbb{R} = g(w_1^{(2)} \cdot a_1^{(1)} + w_2^{(2)} \cdot a_2^{(1)} + w_3^{(2)} \cdot a_3^{(1)} + b^{(2)})$$

$$h_\theta(x) = a^{(2)} \in \mathbb{R} = g(z^{(2)}) = g(w^{(2)} \cdot x + b^{(2)})$$

siendo $w^{(2)} = (w_1^{(2)} \dots w_n^{(2)})$ el conjunto de parámetros para la combinación lineal calculados por la neurona de la capa 2 (también llamados *pesos*), $b^{(2)}$ un término independiente de sesgo (o *bias*), y $g(z)$ la *función de activación*, como la función sigmoide o ReLU.

El *Perceptrón Multicapa* (MLP) es el modelo clásico de red neuronal *feedforward*. Se compone de una capa de entrada, una o más capas ocultas, y una capa de salida. Cada neurona en una capa está completamente conectada a todas las neuronas de la siguiente capa. Generalizando el modelo de una neurona para el MLP se tiene:

$$a^{(1)} \in \mathbb{R}^n = x = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}, \qquad a^{(j)} \in \mathbb{R}^{S_j} = \begin{bmatrix} a_1^{(j)} \\ \vdots \\ a_{s_j}^{(j)} \end{bmatrix} = g(W^{(j)} \cdot a^{(j-1)} + b^{(j)})$$

siendo S_j el número de neuronas en la capa j, y $W^{(j)}$ la matriz de pesos que define el mapeo desde la capa $j-1$ a la capa j y el vector de sesgo $b^{(j)} \in \mathbb{R}^{s_j}$.

El proceso de entrenamiento de una red neuronal consiste en ajustar los pesos ($W^{(j)}$) y sesgos ($b^{(j)}$) para minimizar una *función de costo*, que cuantifica la diferencia entre las predicciones de la red y los valores reales. Este ajuste de parámetros es un problema de optimización que se resuelve mediante algoritmos como el *descenso por gradiente*, usando una técnica llamada *retropropagación* (*backpropagation*).

La *retropropagación* es un algoritmo de aprendizaje supervisado que calcula los gradientes de la función de costo con respecto a los pesos de la red para ajustar sus valores. Este ajuste se realiza iterativamente, actualizando los pesos para que la red minimice la función de costo.

El proceso comienza en la capa de salida de la red, calculando la diferencia entre las predicciones y los valores reales, y luego retropropaga", es decir, propaga este error hacia las capas anteriores, aplicando la regla de la cadena para obtener el gradiente de la función de costo con respecto a los pesos de cada capa. Este procedimiento se repite para cada peso y sesgo en todas las capas de la red, y el proceso completo se ejecuta varias veces sobre el conjunto de datos, durante varias épocas (*epochs*), hasta que se alcanza un mínimo local o la función de costo deja de mejorar.

Se conoce como redes neuronales profundas (*Deep Neural Networks*) aquellas que tienen muchas capas ocultas y son conocidas por su capacidad de capturar patrones complejos en los datos. Existen diferentes arquitecturas para estas redes, como las Redes Neuronales Convolucionales (CNN), las Redes Neuronales Recurrentes (RNN), entre otras.

Las **Redes Neuronales Convolucionales** (CNNs) son especialmente útiles en el procesamiento de imágenes y datos con estructuras espaciales. Las redes convolucionales son simplemente redes neuronales que utilizan convolución en lugar de la multiplicación de matrices en al menos una de sus capas [Goodfellow, 2016].

La operación de convolución se define como:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau) g(t - \tau) d\tau, \tag{2.1}$$

donde f es la entrada y g es el filtro o *kernel*. En el caso discreto, esta operación transforma en una sumatoria.

Las capas típicas de una red convolucional consisten en tres etapas: convolución, activación no lineal y una función de *pooling* [Goodfellow, 2016]. El *pooling* resume las salidas en una región vecina mediante funciones como max pooling o promedio, lo que ayuda a lograr invariancia a pequeñas traslaciones de la entrada, permitiendo que la red sea menos sensible a la ubicación exacta de las características detectadas.

Las **Redes Neuronales Recurrentes** (RNNs) están diseñadas para trabajar con datos secuenciales, como texto, series temporales y señales. A diferencia de las redes feedforward, una RNN tiene conexiones que forman ciclos, permitiendo que la red recuerdeïnformación previa.

La salida de una neurona recurrente en el tiempo t se define como:

$$h_t = f(W_h h_{t-1} + W_x x_t + b), \tag{2.2}$$

donde h_t es el estado oculto en el tiempo t, x_t es la entrada en el tiempo t, W_h y W_x son los pesos, y f es una función de activación. Este mecanismo permite a las RNNs capturar dependencias temporales.

Uno de los principales problemas en redes neuronales es el sobreajuste (*overfitting*), donde el modelo aprende patrones específicos del conjunto de entrenamiento que no se generalizan bien a nuevos datos. Para mitigar este problema, se emplean técnicas de regularización, que buscan reducir el error en los datos de prueba a costa de un posible aumento en el error de entrenamiento. Estrategias comunes incluyen la regularización L_1 y L_2, que penalizan la magnitud de los pesos, y *dropout*, que apaga neuronas de manera aleatoria para mejorar la generalización del modelo.

2.2. *Sparse coding* o codificación esparsa

La *codificación esparsa* es un modelo de factores lineales ampliamente investigado como mecanismo de aprendizaje no supervisado para la extracción y representación de características [Goodfellow, 2016]. Esta técnica no supervisada busca aprender un conjunto de bases sobredimensionadas (*over-complete*) para representar datos de manera eficiente [Ng et al., 2013]. El objetivo es encontrar un conjunto de vectores base ϕ_i, de modo que podamos representar un vector de entrada x como una combinación lineal de estos vectores base:

$$x = \sum_{i=1}^{k} a_i \phi_i \tag{2.3}$$

donde a_i son los coeficientes de la combinación lineal. Mientras que técnicas como el *Análisis de Componentes Principales* (PCA) permiten aprender un conjunto completo de vectores base, la codificación esparsa busca un conjunto sobredimensionado ($k > n$), donde n es la dimensión de la entrada), esto permite capturar mejor las estructuras y patrones inherentes en los datos, pero introduce el desafío de que los coeficientes a_i ya no están determinados de manera única.

Para resolver este problema, la codificación esparsa se añade el criterio de *esparcidad*, lo que implica que solo algunos coeficientes a_i serán distintos de cero o cercanos a cero. Esto es útil porque muchos datos, como las imágenes naturales, pueden describirse como una superposición

de pocos elementos atómicos, como bordes o superficies [Ng et al., 2013]. Este comportamiento también se puede justificar al compararlo con las propiedades de la corteza visual primaria.

El problema de codificación esparsa puede formalizarse como un problema de optimización mediante la definición de la función de costo sobre un conjunto de m vectores de entrada:

$$\min_{a_i^{(j)},\phi_i} \sum_{j=1}^{m} \left\| x^{(j)} - \sum_{i=1}^{k} a_i^{(j)} \phi_i \right\|_2^2 + \lambda \sum_{i=1}^{k} S(a_i^{(j)}) \tag{2.4}$$

donde $S(.)$ es una función de penalización que impone esparcidad en los coeficientes a_i, y λ es una constante que determina la importancia relativa entre el término de reconstrucción y el término de esparcidad. Se puede interpretar que el primer término de la función costo busca una buena reconstrucción de x, mientras que el segundo impone esparcidad, con λ controlando la importancia relativa entre ambos.

Una forma de garantizar esparcidad es mediante la norma L_0 ($S(a_i) = 1(|a_i| > 0)$), que cuenta el número de coeficientes distintos de cero. Sin embargo, esta medida no es diferenciable y resulta difícil de optimizar. En la práctica, se utilizan alternativas como la norma L_1, definida como $S(a_i) = |a_i|$, o la penalización logarítmica, $S(a_i) = \log(1 + a_i^2)$.

La codificación esparsa introduce la suposición de que la mayoría de los coeficientes a_i tienen distribuciones con picos alrededor de cero, lo que justifica el uso de la regularización L_1 o la penalización logarítmica para fomentar esparcidad.

Una de las formas de evitar que la penalización de esparcidad se vuelva arbitrariamente pequeña es imponer restricciones sobre la norma de ϕ_i. La función de costo completa de codificación esparsa, que incluye esta restricción, se formula como:

$$\min_{a_i^{(j)},\phi_i} \sum_{j=1}^{m} \left\| x^{(j)} - \sum_{i=1}^{k} a_i^{(j)} \phi_i \right\|_2^2 + \lambda \sum_{i=1}^{k} S(a_i^{(j)}) \tag{2.5}$$

sujeto a la restricción:

$$\|\phi_i\|_2^2 \leq C, \quad \forall i = 1, \ldots, k. \tag{2.6}$$

Donde C es una constante que impone un límite a la magnitud de los vectores base ϕ_i. En este caso es un problema de optimización con restricciones.

Otra forma de interpretar la codificación esparsa es desde una perspectiva probabilística, modelando los datos como la superposición lineal de características independientes con ruido aditivo (ν). En este caso, la Ecuación 2.3 se reescribe como:

$$x = \sum_{i=1}^{k} a_i \phi_i + \nu(x) \tag{2.7}$$

El objetivo es encontrar un conjunto de vectores base de características ϕ tales que la distribución de las imágenes $P(x \mid \phi)$ esté lo más cerca posible de la distribución empírica de nuestros datos de entrada $P^*(x)$. Generalmente el modelado supone que el ruido sigue una distribución gaussiana

[Goodfellow, 2016] con varianza σ^2, la distribución de los datos x condicionada a los coeficientes a_i y las bases ϕ_i es:

$$P(x \mid a, \phi) = \frac{1}{Z} \exp\left(-\frac{1}{2\sigma^2} \left\| x - \sum_{i=1}^{k} a_i \phi_i \right\|_2^2 \right), \tag{2.8}$$

donde Z es un factor de normalización. Se puede demostrar que mediante este enfoque se obtiene el mismo problema de optimización de la Ecuación 2.4.

El problema de codificación esparsa se formula como la maximización de la probabilidad condicional $p(a, \phi \mid x)$, o de manera equivalente, la minimización de una función de costo que combina el error de reconstrucción y la penalización de esparcidad (Ecuación 2.4).

El aprendizaje en codificación esparsa consiste en dos optimizaciones: una para los coeficientes a_i de cada ejemplo y otra para los vectores base ϕ en conjunto. La optimización de a_i con una penalización L_1 se resuelve como un problema de mínimos cuadrados regularizado, mientras que la de ϕ con una restricción L_2 es un problema de mínimos cuadrados con restricciones cuadráticas. Una desventaja clave es que la codificación de nuevos datos sigue requiriendo optimización, lo que genera un alto costo computacional [Ng et al., 2013].

La codificación esparsa también puede interpretarse como un problema de optimización binivel, similar al enfoque de un *encoder-decoder* que se utiliza en arquitecturas como las *Generative Adversarial Networks* (GANs). En este caso, el proceso de codificación esparsa se puede descomponer en dos tareas: la primera es la codificación, que busca encontrar una representación esparsa h que describa adecuadamente una entrada x mediante una combinación lineal de vectores base, mientras que la segunda tarea es la decodificación, que reconstruye la señal x a partir de la representación esparsa utilizando una matriz de pesos W.

Este proceso puede formularse como un problema de optimización binivel, donde la optimización sobre h busca maximizar la esparcidad mientras minimiza el error de reconstrucción, y la optimización sobre W minimiza el error total de reconstrucción. Esto se expresa matemáticamente como:

$$\min_{W} \max_{h} \left(\lambda \|h\|_1 + \|x - Wh\|_2^2 \right), \tag{2.9}$$

donde $\|h\|_1$ penaliza los coeficientes grandes de h, promoviendo la esparcidad, y $\|x - Wh\|_2^2$ representa el error de reconstrucción. Esta formulación refleja la dinámica de un problema min-max similar al que se encuentra en las GANs, donde el generador intenta minimizar el error de discriminación mientras que el discriminador intenta maximizar su precisión.

La optimización binivel de la codificación esparsa se realiza de forma iterativa. En cada iteración, se optimizan primero los coeficientes esparsos h, manteniendo los vectores base W fijos, resolviendo el siguiente problema de minimización:

$$h^* = \arg\min_{h} \left(\lambda \|h\|_1 + \|x - Wh\|_2^2 \right) \tag{2.10}$$

Luego, se actualizan los vectores base W minimizando el error de reconstrucción global sobre todos los datos de entrenamiento:

$$W^* = \arg\min_W \sum_{i=1}^{N} \|x^{(i)} - Wh^{(i)}\|_2^2 \tag{2.11}$$

Este ciclo de optimización es muy similar al proceso de entrenamiento de un generador y un discriminador en una GAN, donde ambos modelos interactúan iterativamente para mejorar la calidad de la generación y la clasificación. De manera similar, en la codificación esparsa, la interacción entre la optimización de h y W mejora la capacidad del modelo para representar y reconstruir los datos. Este enfoque proporciona una mayor interpretabilidad del proceso de codificación.

2.3. Métodos iterativos de optimización

2.3.1. Alternating Direction Method of Multipliers (ADMM)

ADMM es un algoritmo utilizado para resolver problemas de optimización convexa. La idea principal es dividir estos problemas en problemas más pequeños y manejables. Para lograrlo ADMM combina dos técnicas de optimización: descomposición dual y multiplicadores de Lagrange aumentados. Este algoritmo es utilizado para varios propósitos como la regresión de Lasso, optimización en el procesaminto de imágenes, optimizar redes neuronales de capas grandes de manera distribuida, entre otros. En particular, Monga et al. [2019] hace referencia a la implementación de ADMM-CSNet la cual es una red neuronal profunda que utiliza el algoritmo ADMM para resolver problemas de reconstrucción de señales a partir de muestras submuestreadas, donde Compressed Sensing (CS), proporciona el marco teórico que permite reconstruir la señal a partir de pocos datos, y ADMM es el algoritmo utilizado para resolver los problemas de optimización que surgen en CS.

ADDM combina dos técnicas de optimización:

- **Descomposición dual**: donde el problema se divide en componentes más pequeños que pueden resolverse de forma separada.
- **Multiplicadores de Lagrange aumentados**: utilizados para mejorar la convergencia en problemas donde existen restricciones de igualdad.

Siguiendo con el ejemplo ADMM-CSNet, una generalización del modelo CS puede verse formulado como el siguiente problema de optimización.

$$\min_x \frac{1}{2}\|\Phi x - y\|_2^2 + \sum_{i=1}^{C} \lambda_i g_i(D_i x) \tag{2.12}$$

Donde:

- x es la señal a ser reconstruida.
- Φ es la matriz de muestreo que transforma la señal original.
- y es el vector de observaciones submuestreadas.
- λ_i son coeficientes de regularización positivos que controlan el grado de ajuste de cada término regularizador.
- $g_i(\cdot)$ es una función que induce esparcidad, típicamente la norma ℓ_1, que fomenta la obtención de soluciones parciales esparsas.

- $\{D_i\}_{i=1}^{C}$ son operadores de Toeplitz que actúan como filtros lineales sobre la señal x. Un ejemplo común de D_i son transformaciones tipo wavelet, útiles para descomponer señales en diferentes escalas.

El primer término de la función objetivo, $\frac{1}{2}\|\Phi x - y\|_2^2$, representa el ajuste de la señal reconstruida x a las observaciones y. El segundo término es una suma ponderada de funciones $g_i(D_i x)$, que impone regularización en la reconstrucción de x en diferentes dominios definidos por los operadores D_i, promoviendo así soluciones con ciertas propiedades, como esparcidad o suavidad.

Para resolver este problema mediante ADMM, se introduce una variable auxiliar $z_i = D_i x$ que permite reformular la función objetivo y aplicar un esquema iterativo de actualización para las variables x, z_i y los multiplicadores de Lagrange. Este proceso es eficiente, pues cada subproblema resultante se resuelve de manera más sencilla que el problema original. La combinación de ADMM con el modelo de *Compressed Sensing* ha demostrado ser una herramienta poderosa para la reconstrucción de señales a partir de datos submuestreados, logrando un equilibrio entre precisión y eficiencia computacional.

Ventajas

- **Descomposición del problema**: ADMM permite dividir problemas grandes y complejos en subproblemas más simples y manejables, lo que facilita su solución.
- **Convergencia robusta**: ADMM garantiza la convergencia a una solución óptima en muchos problemas convexos bajo condiciones generales.
- **Eficiencia en problemas distribuidos**: ADMM es especialmente útil en entornos distribuidos o paralelos, ya que los subproblemas pueden resolverse de forma independiente. Esto lo hace ideal para aplicaciones en redes grandes, como las redes neuronales distribuidas.
- **Aplicabilidad a problemas con restricciones**: ADMM maneja eficientemente problemas con restricciones de igualdad, como aquellos en programación convexa o procesamiento de señales.
- **Flexibilidad**: ADMM puede adaptarse a diferentes tipos de funciones objetivo y restricciones, incluyendo aquellas con funciones no diferenciables, como la regularización ℓ_1.

Desventajas

- **Lenta convergencia en problemas no convexos**: Aunque ADMM garantiza la convergencia en problemas convexos, su rendimiento puede ser más lento o incluso no converger en problemas no convexos.
- **Sensibilidad a la elección de parámetros**: ADMM requiere ajustar parámetros como el parámetro de penalización ρ. Una mala elección de este parámetro puede ralentizar la convergencia o afectar el rendimiento.
- **Iteraciones costosas**: A pesar de que cada iteración resuelve subproblemas más simples, estos subproblemas pueden seguir siendo costosos, especialmente cuando se manejan matrices grandes o operadores complejos.
- **Necesidad de sincronización**: En aplicaciones distribuidas, ADMM puede requerir sincronización entre nodos o máquinas, lo que introduce latencia y sobrecarga de comunicación en sistemas muy grandes o heterogéneos.

2.3.2. Iterative Shrinkage-Thresholding Algorithm

Un método comúnmente utilizado para resolver el problema de codificación esparsa es el *terative Shrinkage-Thresholding Algorithm* (ISTA, por sus siglas en inglés). Este algoritmo es eficiente para problemas con regularización L_1, ya que combina un paso de descenso de gradiente con una operación de umbralización suave para imponer esparcidad.

Se debe en encontrar un código esparso $\mathbf{x} \in \mathbb{R}^m$ tal que $\mathbf{y} \approx W\mathbf{x}$, donde $W \in \mathbb{R}^{n \times m}$ es un diccionario sobredeterminado, y $\mathbf{y} \in \mathbb{R}^n$ es un vector de entrada [Monga et al., 2019]. Para fomentar la esparcidad, se busca que el número de coeficientes de $\mathbf{x}$ que son diferentes de cero sea el menor posible. Este problema puede formularse como una optimización convexa:

$$\min_{\mathbf{x} \in \mathbb{R}^m} \frac{1}{2} \|\mathbf{y} - W\mathbf{x}\|_2^2 + \lambda \|\mathbf{x}\|_1 \tag{2.13}$$

donde $\lambda > 0$ es un parámetro de regularización que controla la esparcidad de la solución.

En su forma más simple, las iteraciones de ISTA se definen como:

$$\mathbf{x}^{l+1} = S_\lambda \left\{ \left(\mathbf{I} - \frac{1}{\mu} W^T W \right) \mathbf{x}^l + \frac{1}{\mu} W^T \mathbf{y} \right\}, \quad l = 0, 1, \ldots, \tag{2.14}$$

donde $\mathbf{I} \in \mathbb{R}^{m \times m}$ es la matriz identidad, y $S_\lambda(\cdot)$ es el operador de *soft-thresholding*, definido elemento a elemento como:

$$S_\lambda(x) = \text{sign}(x) \cdot \text{máx}\{|x| - \lambda, 0\}. \tag{2.15}$$

El algoritmo ISTA tiene las siguientes etapas:

Inicialización: Se empieza con un valor inicial para $x^{(0)}$.

Iteración: para $x^{(l+1)}$ se computa la Ecuación 2.14. Repetir el proceso hasta que la diferencia entre iteraciones sucesivas sea menor que un umbral dado.

El ISTA es ampliamente utilizado en una variedad de campos, por ejemplo restauración de imágenes, eliminación de ruido en señales, compresión, etc

Ventajas:

- **Simplicidad de implementación:** ISTA es fácil de implementar debido a su estructura iterativa simple y a que utiliza operaciones básicas de descenso de gradiente y umbralización.
- **Favorece la esparsidad:** Gracias a la regularización L_1, ISTA promueve soluciones esparsas, lo que es útil en aplicaciones donde solo un subconjunto pequeño de variables es relevante, como en la regresión Lasso o la reconstrucción de señales.
- **Generalizable:** Se puede aplicar a una variedad de problemas de optimización convexa, especialmente en aquellos donde se busca minimización con restricciones de esparsidad.

Desventajas:

- **Convergencia lenta:** Una de las principales desventajas del ISTA es que puede converger lentamente, especialmente cuando se necesita una solución de alta precisión.

- **Requiere buen ajuste del tamaño de paso:** La velocidad de convergencia y la precisión dependen de la elección adecuada del tamaño de paso (t). Si el tamaño de paso es demasiado grande, el algoritmo puede no converger, y si es demasiado pequeño, puede ser extremadamente lento.
- **Puede no ser adecuado para problemas no convexos:** Aunque ISTA es eficiente en problemas convexos, no garantiza encontrar el óptimo global en problemas no convexos, donde pueden surgir múltiples mínimos locales.

2.3.3. Fast Iterative Shrinkage-Thresholding Algorithm

FISTA (*Fast Iterative Shrinkage-Thresholding Algorithm*) es una mejora de ISTA para resolver problemas de optimización convexa con regularización, como los problemas de regresión LASSO o el problema de reconstrucción en imágenes. FISTA introduce una aceleración en la convergencia, lo que lo hace más eficiente que métodos anteriores, como ISTA.

La aceleración de FISTA se basa en la idea de momentum, que se inspira en los métodos de aceleración de Nesterov. En lugar de actualizar directamente las soluciones usando solo el gradiente, FISTA utiliza una combinación de las soluciones actuales y previas para acelerar la convergencia.

$$\min_x \left(f(x) + g(x)\right) \tag{2.16}$$

Donde:

- $f(x)$ es una función convexa y diferenciable con gradiente de Lipschitz constante L.
- $g(x)$ es una función convexa (posiblemente no diferenciable), como la penalización L1 en el caso del LASSO.

Algoritmo FISTA

El algoritmo FISTA sigue los siguientes pasos:

1. **Inicialización**: Se eligen los valores iniciales x_0, y_1 y el parámetro $t_1 = 1$. Se escoge un valor para L, la constante de Lipschitz del gradiente de f.
2. **Iteración**: Para cada iteración $k \geq 1$:
   a) Calcular el gradiente de f en y_k:

   $$x_k = \text{prox}_{g,\frac{1}{L}}\left(y_k - \frac{1}{L}\nabla f(y_k)\right) \tag{2.17}$$

   Donde $\text{prox}_{g,\frac{1}{L}}$ es el operador de proximidad para manejar la parte no diferenciable $g(x)$.

   b) Actualizar el parámetro de aceleración t_{k+1}:

   $$t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2} \tag{2.18}$$

   c) Actualizar el punto y_{k+1}:

   $$y_{k+1} = x_k + \frac{t_k - 1}{t_{k+1}}(x_k - x_{k-1}) \tag{2.19}$$

3. Repetir hasta que se alcance la convergencia o se llegue al número máximo de iteraciones.

Ventajas

- **Convergencia rápida**: FISTA tiene una tasa de convergencia de $O(1/k^2)$, en comparación con ISTA que tiene una tasa de $O(1/k)$.

2.4. *Algorithm Unrolling*

El *desenrollado de algoritmos* (o *Algorithm Unrolling* en inglés) es un enfoque que busca representar algoritmos iterativos como redes neuronales profundas, donde cada iteración del algoritmo se corresponde con una capa de la red. Este enfoque permite que los algoritmos sean más interpretables y eficientes, al mismo tiempo que mantienen la capacidad de ser entrenados con datos del mundo real [Monga et al., 2019], lo que representa una ventaja sobre los algoritmos iterativos tradicionales.

Se presentó por primera vez por Gregor and LeCun [2010] en su artículo *Learning Fast Approximations of Sparse Coding*, que presenta la técnica conocida como *algorithm unrolling* o *unfolding*, que establece una conexión entre los algoritmos iterativos, como los utilizados en *sparse coding*, y las arquitecturas de redes neuronales. En los últimos años, ha habido un aumento significativo en los esfuerzos por desenrollar algoritmos iterativos para abordar problemas importantes en el procesamiento de señales e imágenes. En la Sección 3 se presentan algunos trabajos relevantes en el área.

A diferencia de los enfoques iterativos convencionales, donde los modelos y las condiciones previas se diseñan a partir de un análisis detallado de los procesos físicos involucrados, las técnicas de aprendizaje profundo intentan descubrir automáticamente la información del modelo a través de la optimización de los parámetros de la red, los cuales se aprenden a partir de datos del mundo real. Esta capacidad permite a las redes profundas superar las limitaciones del modelo, especialmente cuando el escenario físico subyacente es difícil de caracterizar con precisión [Monga et al., 2019].

La idea básica detrás del *desenrollado* es que, en lugar de ejecutar un algoritmo iterativo hasta la convergencia, se desenrolla el proceso en un número finito de pasos y se representa cada paso como una capa de una red neuronal. Concatenando estas capas, se forma una red neuronal profunda (ver Figura 2.1). Al pasar un conjunto de datos a través de esta red, se simula la ejecución de un algoritmo de optimización iterativo durante un número determinado de iteraciones [Monga et al., 2019]. La arquitectura de esta red puede entrenarse utilizando *backpropagation*, al igual que las ANN, permitiendo que los parámetros del modelo, como los coeficientes de regularización y los parámetros de la capa, se aprendan directamente a partir de los datos.

El entrenamiento adecuado de las redes desenrolladas es crucial para aprovechar todo su potencial. A diferencia de las redes neuronales convencionales, las redes desenrolladas a pueden presentar estructuras personalizadas que requieren estrategias de entrenamiento específicas. Además, las redes desenrolladas pueden compartir parámetros entre capas, lo que las asemeja a las redes neuronales recurrentes (RNNs), conocidas por su dificultad para ser entrenadas correctamente [Monga et al., 2019]. Debido a estas características, algunas técnicas existentes aplican esquemas de preentrenamiento capa por capa para mejorar el rendimiento.

Una de las ventajas del desenrollado de algoritmos es que ofrece una solución al problema de la falta de interpretabilidad en las redes neuronales convencionales, permitiendo que las redes resultantes se interpreten como algoritmos optimizados por parámetros, varias investigaciones se han realizado en este sentido, relacionando arquitecturas conocidas con el proceso de desenrollado de algún algoritmo

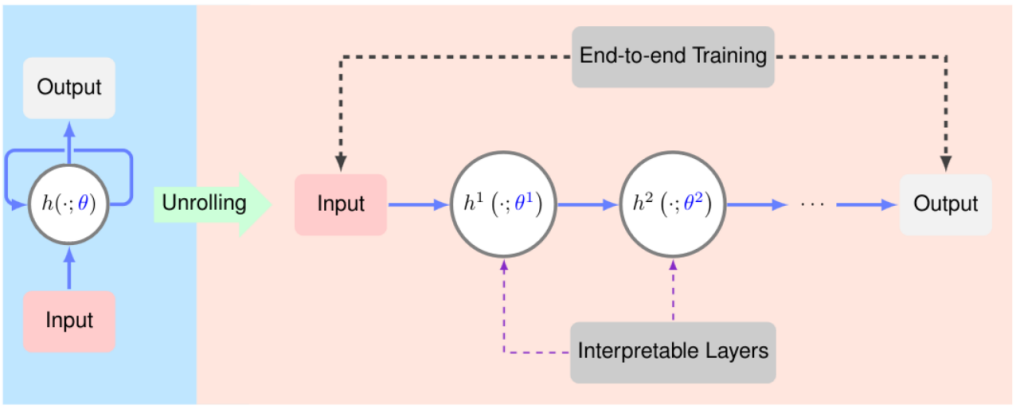


Figura 2.1: Esquema de *algorithm unrolling*: dado un algoritmo iterativo (izquierda), se puede generar una red profunda correspondiente (derecha) encadenando sus iteraciones h. Los parámetros entrenables están coloreados en azul. Tomado de Monga et al. [2019]

iterativo. Esta propiedad es especialmente ventajosa en aplicaciones de procesamiento de señales e imágenes, donde la incorporación de conocimiento sobre la estructura inherente del problema puede ayudar a diseñar redes más eficientes [Monga et al., 2019].

En comparación con las redes neuronales convencionales, las redes desenrolladas contienen menos parámetros de entrenamiento, lo que les permite ser más eficientes y ofrecer una mejor capacidad de generalización, especialmente cuando se tienen datos de entrenamiento limitados. Sin embargo, las redes desenrolladas pueden sufrir de problemas de sobreajuste si no se entrenan adecuadamente, debido a la falta de regularización explícita en algunos casos.

2.4.1. Aplicaciones

LISTA

Un ejemplo concreto de *Algorithm Unrolling* es el desenrollado del algoritmo *Iterative Shrinkage-Thresholding Algorithm* (ISTA), presentado por Gregor and LeCun [2010]. En este caso, cada iteración de ISTA se convierte en una capa de una red neuronal, y la red resultante se entrena utilizando un conjunto de datos de entrenamiento real mediante *backpropagation*. Este enfoque da lugar a lo que se conoce como *Learned ISTA* (LISTA) [Monga et al., 2019].

Como se explica en la Sección 2.3.2, ISTA es un algoritmo iterativo para resolver el problema de optimización convexa (Ecuación 2.13) donde cada paso de la iteración resuelve la Ecuación 2.14. Para desenrollar ISTA en una red, cada paso de iteración se puede asignar a una capa de la neurona, como se muestra en la Figura 2.2. Ejecutar el algoritmo ISTA L veces, se puede interpretar como concatenar L capas de la red.

La red desenrollada se entrena con conjuntos de datos reales para optimizar los parámetros W_t, W_e y λ. La versión aprendida, llamada *LISTA*, puede lograr una mayor eficiencia en comparación con *ISTA*, siendo particularmente útil cuando $\mathbf{W}$ no se conoce con exactitud [Monga et al., 2019]. El entrenamiento se realiza utilizando una secuencia de vectores $\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_N \in \mathbb{R}^n$ y sus correspondientes códigos esparsos de referencia (*groundtruth*) $\mathbf{x}_1^*, \mathbf{x}_2^*, \ldots, \mathbf{x}_N^*$.

Al alimentar cada $\mathbf{y}_n$ a la red, se obtiene la salida $\hat{\mathbf{x}}_n(\mathbf{y}_n; W_t, W_e, \lambda)$, que representa el código

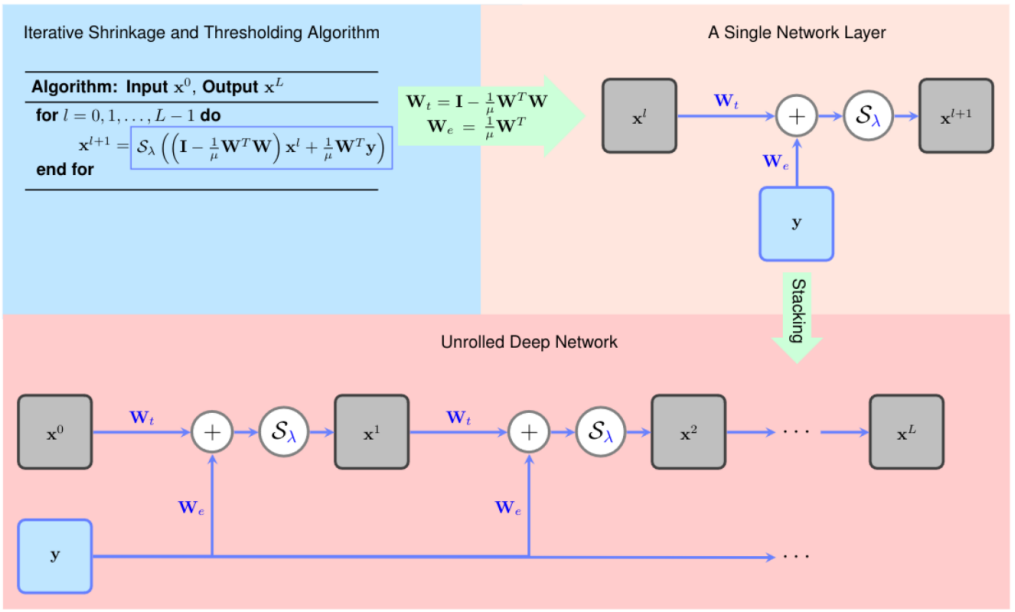


Figura 2.2: Ilustración del *Learned ISTA*. Tomado de Monga et al. [2019]

esparso predicho para $\mathbf{y}_n$. Comparando este valor con el código esparso verdadero $\mathbf{x}_n^*$, se define la función de pérdida del entrenamiento de la red como:

$$\mathcal{L}(W_t, W_e, \lambda) = \frac{1}{N}\sum_{n=1}^{N} \|\hat{\mathbf{x}}_n(\mathbf{y}_n; W_t, W_e, \lambda) - \mathbf{x}_n^*\|_2^2 \tag{2.20}$$

La red se entrena mediante la minimización de esta función de pérdida, utilizando técnicas populares de aprendizaje basadas en gradientes, como el *descenso por gradiente estocástico* (SGD). Los estudios empíricos han demostrado que el número de capas L en *LISTA* puede ser significativamente menor en comparación con el número de iteraciones necesarias en *ISTA* para lograr la convergencia con una nueva entrada observada.

ADMM-CSNet

Yang et al. [2020] desenrollan el algoritmo de ADMM, presentado en la Sección 2.3.1, en una red profunda llamada **ADMM-CSNet**, enfocada en la resolución de problemas de *Compressive Sensing* (CS).

Dado un conjunto de mediciones lineales aleatorias $\mathbf{y} \in \mathbb{C}^m$, formado por $\mathbf{y} \approx \Phi\mathbf{x}$, donde $\Phi \in \mathbb{C}^{m\times n}$ es una matriz de medición y $m < n$, *Compressive Sensing* (CS) busca reconstruir la señal original $\mathbf{x} \in \mathbb{R}^n$ aprovechando su estructura esparsa en un dominio transformado. El modelo generalizado de CS se puede formular como el siguiente problema de optimización:

$$\min_{\mathbf{x}} \frac{1}{2}\|\Phi\mathbf{x} - \mathbf{y}\|_2^2 + \sum_{i=1}^{C} \lambda_i g(D_i\mathbf{x}) \tag{2.21}$$

donde λ_i son coeficientes de regularización positivos, $g(\cdot)$ es una función que fomenta la esparcidad, y $\{D_i\}_{i=1}^{C}$ es una secuencia de operadores de Toeplitz[1] que efectivamente realizan operaciones de filtrado lineal. Concretamente, D_i puede ser interpretado como una transformada *wavelet*[2], y g se puede escoger como la norma ℓ_1.

Para resolver este problema, se puede aplicar el método ADMM, el cuál se se desenrolla en una red profunda denominada **ADMM-CSNet**, donde se concatenan las iteraciones del método a lo largo de varias capas y se aprenden los parámetros $\lambda_i, \rho_i, \eta_i, D_i$ en cada capa. La red se entrena minimizando una versión normalizada del error cuadrático medio (RMSE). La Figura 2.3 presenta un diagrama del desenrollado del método ADMM.

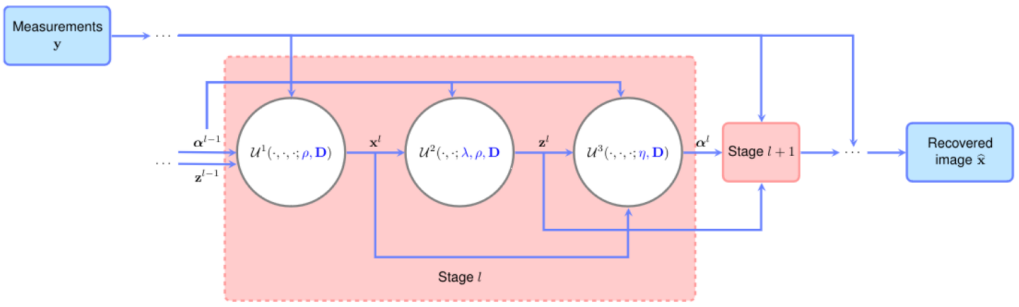


Figura 2.3: Representación de ADMM-CSNet. Tomada de [Monga et al., 2019]

3. Breve revisión del Estado del arte

El trabajo de Metz et al. [2016] presenta una técnica para estabilizar las redes generativas adversarias (GANs) a través de la aplicación de *unrolling* del objetivo del generador, tomando en cuenta una optimización del discriminador. Esta técnica permite ajustar el entrenamiento entre usar un discriminador óptimo, lo cual es ideal pero no es eficiente en la práctica, y el uso del discriminador actual, que puede ser inestable. El método propuesto presenta solución problemas comunes de las GANs, como el colapso de modos y la oscilación entre generador y discriminador. La metodología se probó en varias tareas de procesamiento de imágenes, demostrando mejoras significativas en la estabilidad del entrenamiento y en la cobertura de modos de la distribución de datos, reduciendo así los problemas de colapso típicos de las GANs convencionales.

Li et al. [2019] proponen un enfoque basado en el desenrollado de algoritmos para abordar el problema del desenfoque en imágenes (*deblurring*), específicamente el desenfoque causado por movimiento de cámara. Utilizando una técnica de regularización de variación total (*total variation denoising*) en el dominio de gradiente, se desarrolla un algoritmo iterativo que luego es desenrollado para formar una red neuronal profunda. Esta red neuronal, denominada *Deblurring via Algorithm Unrolling* (DAU), optimiza los parámetros del algoritmo a través del entrenamiento supervisado. Los resultados que obtuvieron demuestran que el enfoque DAU no solo supera en precisión a los métodos iterativos y redes neuronales previas, sino que también conserva la interpretabilidad del modelo. Los experimentos muestran que DAU es capaz de recuperar detalles finos y minimizar los artefac-

[1] Los operadores de Toeplitz son matrices que tienen la propiedad de que cada una de sus diagonales tiene elementos constantes.

[2] La transformada wavelet descompone una señal proporcionando una representación tanto en el dominio del tiempo como en el dominio de la frecuencia.

tos en imágenes desenfocadas, obteniendo mejores puntuaciones en métricas como PSNR, ISNR y SSIM en comparación con los métodos actuales de referencia.

Un enfoque para la codificación esparsa rápida aplicada al reconocimiento de objetos es presentado por Sun and Yu [2021]. El método propuesto utiliza una red neuronal con una sola capa oculta (SLNN) para aproximar la representación esparsa de las muestras de prueba. En la fase de entrenamiento, las características óptimas de las muestras se calculan mediante el algoritmo de codificación esparsa, y luego estas características se usan como *groundtruth* para entrenar la SLNN. Una vez entrenada, esta red puede estimar rápidamente las características esparsas de nuevas muestras, lo que es fundamental para aplicaciones en tiempo real. El enfoque propuesto fue validado en diez conjuntos de datos de referencia, así como en dos conjuntos de imágenes faciales. Los resultados experimentales demuestran que este método logra un error cuadrático medio (RMSE) significativamente menor que otros métodos de aproximación rápida basados en redes profundas, mientras que mantiene un alto rendimiento en tareas de reconocimiento de objetos. Además, el tiempo de prueba se reduce considerablemente, lo que lo convierte en una opción viable para aplicaciones de reconocimiento en tiempo real con conjuntos de datos de pequeña escala.

En el trabajo de Shi et al. [2021], los autores abordan el problema de *Joint Activity Detection and Channel Estimation* (JADCE) para acceso masivo en redes IoT. Este desafío surge en el acceso aleatorio sin concesión, donde un gran número de dispositivos se comunican de manera esporádica, lo que lleva a una alta demanda computacional. El problema se formula como una tarea de estimación de matrices dispersas por grupos. Para resolverlo, los autores proponen un marco de *unrolling* de algoritmos basado en el *Iterative Shrinkage-Thresholding Algorithm* (ISTA), que se mapea en una red neuronal recurrente (RNN). A través de un análisis teórico, se demuestra que la red neuronal desenrollada propuesta ofrece una mayor robustez y eficiencia en comparación con los métodos tradicionales de *Compressed Sensing*.

[An et al., 2022] presentan una red neuronal interpretable llamada **NISTA-Net** para el diagnóstico de fallos mecánicos, basada en el desenrollado del algoritmo NISTA para un modelo de codificación dispersa convolucional en múltiples capas (ML-CSC). NISTA-Net destaca por su capacidad de extraer características de fallos en señales de vibración mediante un proceso de visualización de características. Esto permite que los usuarios comprendan mejor cómo la red realiza las clasificaciones, lo que mejora su credibilidad en aplicaciones de diagnóstico de fallos. Los resultados experimentales muestran que **NISTA-Net** supera a otros métodos de redes neuronales, tales como *ResNet-18* y *WaveletKernelNet*, en precisión de diagnóstico y robustez frente a interferencias de ruido. Finalmente, los autores proponen un enfoque sistemático para el diseño de redes de diagnóstico de fallos interpretables, lo cual podría inspirar futuras investigaciones en el campo.

[Xiao et al., 2023] proponen una nueva variante del autoencoder variacional basada en codificación esparsa, denominada SC-VAE. Este enfoque integra la codificación esparsa con el algoritmo LISTA (desenrollado de ISTA), permitiendo así aprender representaciones esparsas en el espacio latente. Los experimentos realizados en dos conjuntos de datos de imágenes muestran que SC-VAE supera a los métodos de última generación en la tarea de reconstrucción de imágenes. Además, el uso de códigos esparsos permite abordar tareas posteriores como la generación de imágenes y la segmentación no supervisada a través de la agrupación de parches de imágenes.

4. Aplicación

En esta sección se presenta un caso práctico de aplicación de la técnica de desenrollado de algoritmo, sobre un conjunto de datos reales.

En construcción...

5. Conclusiones

Referencias

Botao An, Shibin Wang, Zhibin Zhao, Fu Qin, Ruqiang Yan, and Xuefeng Chen. Interpretable neural network via algorithm unrolling for mechanical fault diagnosis. *IEEE Transactions on Instrumentation and Measurement*, 71:1–11, 2022. URL https://api.semanticscholar.org/CorpusID:250366412.

Ian Goodfellow. *Deep learning*. MIT press, 2016.

Karol Gregor and Yann LeCun. Learning fast approximations of sparse coding. In *International Conference on Machine Learning*, 2010. URL https://api.semanticscholar.org/CorpusID:13333501.

Yuelong Li, Mohammad-Reza Tofighi, Vishal Monga, and Yonina C. Eldar. An algorithm unrolling approach to deep image deblurring. *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 7675–7679, 2019. URL https://api.semanticscholar.org/CorpusID:61153486.

Luke Metz, Ben Poole, David Pfau, and Jascha Narain Sohl-Dickstein. Unrolled generative adversarial networks. *ArXiv*, abs/1611.02163, 2016. URL https://api.semanticscholar.org/CorpusID:6610705.

Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997. ISBN 0070428077.

Vishal Monga, Yuelong Li, and Yonina C. Eldar. Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing. *IEEE Signal Processing Magazine*, 38:18–44, 2019. URL https://api.semanticscholar.org/CorpusID:209444725.

Andrew Ng, Jiquan Ngiam, Chuan Yu Foo, Yifan Mai, Caroline Suen, Adam Coates, Andrew Maas, Awni Hannun, Brody Huval, Tao Wang, and Sameep Tandon. Deep learning tutorial. Computer Science Department, Stanford University, 2013. URL http://ufldl.stanford.edu/tutorial/. Material contributed by: Andrew Ng et al.

Yandong Shi, Hayoung Choi, Yuanming Shi, and Yong Zhou. Algorithm unrolling for massive access via deep neural networks with theoretical guarantee. *IEEE Transactions on Wireless Communications*, 21:945–959, 2021. URL https://api.semanticscholar.org/CorpusID:235489765.

Zhenzhen Sun and Yuanlong Yu. Fast approximation for sparse coding with applications to object recognition. *Sensors (Basel, Switzerland)*, 21, 2021. URL https://api.semanticscholar.org/CorpusID:232098252.

Pan Xiao, Peijie Qiu, Sung Min Ha, Abdalla Bani, Shuang Zhou, and Aristeidis Sotiras. Sc-vae: Sparse coding-based variational autoencoder with learned ista. *Available at SSRN 4794775*, 2023.

Yan Yang, Jian Sun, Huibin Li, and Zongben Xu. Admm-csnet: A deep learning approach for image compressive sensing. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42:521–538, 2020. URL https://api.semanticscholar.org/CorpusID:54560071.